CYBER SECURITY
PRACTICAL CODES

Practicals 1 to 10

Codes and command sequences arranged practical-wise, with the original code titles.

PRACTICAL 1 — SUBSTITUTION AND TRANSPOSITION CIPHER

1. Caesar Cipher

```
def encrypt(text, key):
    cipher = ""
    for ch in text:
        if ch.isalpha():
            cipher += chr((ord(ch) - 65 + key) % 26 + 65)
        else:
            cipher += ch
    return cipher

def decrypt(cipher, key):
    text = ""
    for ch in cipher:
        if ch.isalpha():
            text += chr((ord(ch) - 65 - key) % 26 + 65)
        else:
            text += ch
    return text

msg = input("Enter message: ").upper()
key = int(input("Enter key: "))

cipher = encrypt(msg, key)
print("Encrypted Message:", cipher)

print("Decrypted Message:", decrypt(cipher, key))
```

2. Transposition Cipher

```
def encrypt(text, cols):
    while len(text) % cols != 0:
        text += 'X'

    cipher = ""
    for i in range(cols):
        for j in range(i, len(text), cols):
            cipher += text[j]

    return cipher

msg = input("Enter message: ").upper()
cols = int(input("Enter columns: "))

cipher = encrypt(msg, cols)
print("Encrypted Message:", cipher)
```

PRACTICAL 2 — RSA

1. RSA Simple Version

```
# RSA Simple Version

p = int(input("Enter p (prime): "))
q = int(input("Enter q (prime): "))

n = p * q
phi = (p - 1) * (q - 1)

# choose e
e = 3
while (phi % e == 0):
    e += 2

# find d
d = 1
while (d * e) % phi != 1:
    d += 1

print("Public Key:", (e, n))
print("Private Key:", (d, n))

# encryption
msg = int(input("Enter number message: "))
cipher = (msg ** e) % n
print("Encrypted:", cipher)

# decryption
plain = (cipher ** d) % n
print("Decrypted:", plain)
```

PRACTICAL 3 — MESSAGE AUTHENTICATION CODES

1. HMAC-SHA256

```
import hmac
import hashlib

# Secret key shared between sender and receiver
key = b"secretkey"

# Message
message = input("Enter message: ")

# Generate MAC
mac = hmac.new(key, message.encode(), hashlib.sha256).hexdigest()

print("Generated MAC:", mac)

# Verification
received_message = input("Enter received message for verification: ")

received_mac = hmac.new(
    key,
    received_message.encode(),
    hashlib.sha256
).hexdigest()

if mac == received_mac:
    print("Message Verified")
else:
    print("Message Tampered")
```

2. SHA-256 Hash

```
import hashlib

# Message
message = input("Enter message: ")

# Generate Hash
hash_value = hashlib.sha256(message.encode()).hexdigest()

print("Generated Hash:", hash_value)

# Verification
received_message = input("Enter received message for verification: ")

received_hash = hashlib.sha256(received_message.encode()).hexdigest()

if hash_value == received_hash:
    print("Message Integrity Verified")
else:
    print("Message Tampered")
```

3. CMAC

```
pip install pycryptodome

from Crypto.Hash import CMAC
from Crypto.Cipher import AES

# Secret key (16 bytes)
key = b"1234567890123456"
message = input("Enter message: ")

cobj = CMAC.new(key, ciphermod=AES)
cobj.update(message.encode())
mac = cobj.hexdigest()

print("Generated CMAC:", mac)
```

4. GMAC (AES-GCM)

```
pip install cryptography

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

key = AESGCM.generate_key(bit_length=128)
aesgcm = AESGCM(key)
nonce = os.urandom(12)
message = input("Enter message: ").encode()
ciphertext = aesgcm.encrypt(nonce, message, None)

print("Generated GMAC/AES-GCM Authentication Successful")
```

5. CBC-MAC

```
pip install pycryptodome

from Crypto.Hash import CMAC
from Crypto.Cipher import AES

key = b"1234567890123456"
message = input("Enter message: ")

cbc_mac = CMAC.new(key, ciphermod=AES)
cbc_mac.update(message.encode())

print("Generated CBC-MAC:", cbc_mac.hexdigest())
```

PRACTICAL 4 — DIGITAL SIGNATURES

1. RSA Digital Signature (Simple Version)

```
# RSA Digital Signature (Simple Version)

p = 3
q = 11

n = p * q
phi = (p - 1) * (q - 1)

# Choose e
e = 3

# Find d
d = 1
while (d * e) % phi != 1:
    d += 1

print("Public Key:", (e, n))
print("Private Key:", (d, n))

# Message
msg = int(input("Enter message (number less than n): "))

# Signing (using private key)
signature = (msg ** d) % n

print("Digital Signature:", signature)

# Verification (using public key)
verified = (signature ** e) % n

print("Verified Message:", verified)

if verified == msg:
    print("Signature Valid")
else:
    print("Signature Invalid")
```

PRACTICAL 5 — DIFFIE-HELLMAN KEY EXCHANGE

1. Diffie-Hellman Key Exchange

```
# Diffie-Hellman Key Exchange

P = int(input("Enter Prime Number (P): "))
G = int(input("Enter Primitive Root (G): "))

# Alice's private key
a = int(input("Enter Alice Private Key: "))

# Bob's private key
b = int(input("Enter Bob Private Key: "))

# Public keys
A = (G ** a) % P
B = (G ** b) % P

print("\nAlice Public Key:", A)
print("Bob Public Key:", B)

# Shared Secret Key
keyA = (B ** a) % P
keyB = (A ** b) % P

print("\nSecret Key calculated by Alice:", keyA)
print("Secret Key calculated by Bob:", keyB)

if keyA == keyB:
    print("\nKey Exchange Successful")
else:
    print("\nKey Exchange Failed")
```

PRACTICAL 6 — IP SECURITY (IPSEC) CONFIGURATION

1. Router Interface Configuration

```
R1
interface fa0/0
ip address 10.0.0.1 255.255.255.0
no shutdown

R2
interface fa0/0
ip address 10.0.0.2 255.255.255.0
no shutdown
```

2. ISAKMP Policy — R1

```
enable
configure terminal

crypto isakmp policy 10
encryption aes
hash sha
authentication pre-share
group 2
exit
```

3. ISAKMP Policy — R2

```
enable
configure terminal

crypto isakmp policy 10
encryption aes
hash sha
authentication pre-share
group 2
exit
```

4. Shared Password

```
R1
crypto isakmp key cisco123 address 10.0.0.2

R2
crypto isakmp key cisco123 address 10.0.0.1
```

5. Transform Set

```
R1
crypto ipsec transform-set MYSET esp-aes esp-sha-hmac

R2
crypto ipsec transform-set MYSET esp-aes esp-sha-hmac

crypto ipsec transform-set MYSET ?
```

6. ACL

```
R1
access-list 101 permit ip any any

R2
access-list 101 permit ip any any
```

7. Crypto Map

```
R1
crypto map VPNMAP 10 ipsec-isakmp
set peer 10.0.0.2
set transform-set MYSET
match address 101
exit

R2
```

```
crypto map VPNMAP 10 ipsec-isakmp
set peer 10.0.0.1
set transform-set MYSET
match address 101
exit
```

8. Apply Crypto Map

```
R1
interface fa0/0
crypto map VPNMAP
exit

R2
interface fa0/0
crypto map VPNMAP
exit
```

9. Verification Commands

```
show crypto isakmp sa
show crypto ipsec sa
show crypto map
```

Practical 7–10: Topology, Connections & Configuration

Supplementary setup guide added to the Cyber Security Practicals 1–10 codes PDF.

PRACTICAL NO. 7 — WEB SECURITY WITH SSL/TLS

Topology

```
PC0  ----------------  2960 Switch  ----------------  Server0
       Copper Straight-Through             Copper Straight-Through
```

Connections

From device / port	To device / port	Cable
PC0 FastEthernet0	Switch (any FastEthernet port)	Copper Straight-Through
Switch (any FastEthernet port)	Server0 FastEthernet0	Copper Straight-Through

IP configuration

Device	IP address	Subnet mask
Server0	192.168.1.2	255.255.255.0
PC0	192.168.1.10	255.255.255.0

Configuration steps

```
SERVER0
1. Desktop → IP Configuration
   IP Address: 192.168.1.2
   Subnet Mask: 255.255.255.0

2. Services → HTTP
   HTTP: On
   HTTPS: On

PC0
1. Desktop → IP Configuration
   IP Address: 192.168.1.10
   Subnet Mask: 255.255.255.0

2. Desktop → Web Browser
   Open: https://192.168.1.2
```

Verification: Open the HTTPS URL from PC0. If the webpage opens through HTTPS, the practical treats SSL/TLS as active. Packet Tracer automatically generates a certificate when HTTPS is enabled.

PRACTICAL NO. 8 — INTRUSION DETECTION SYSTEM (IDS)

Topology

```
PC0  ----------------  Router 2811  ----------------  Server0
                         Fa0/0       Fa0/1

PC0:     192.168.1.10                Server0: 192.168.2.10
Router:  192.168.1.1 (Fa0/0)         Router:  192.168.2.1 (Fa0/1)
```

Connections

From device / port	To device / port	Cable
PC0 FastEthernet0	Router Fa0/0	Copper Straight-Through
Router Fa0/1	Server0 FastEthernet0	Copper Straight-Through

IP configuration

Device / interface	IP address	Subnet mask	Default gateway
PC0 FastEthernet0	192.168.1.10	255.255.255.0	192.168.1.1
Router Fa0/0	192.168.1.1	255.255.255.0	—
Router Fa0/1	192.168.2.1	255.255.255.0	—
Server0 FastEthernet0	192.168.2.10	255.255.255.0	192.168.2.1

Router configuration

```
enable
configure terminal

interface fa0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit

interface fa0/1
ip address 192.168.2.1 255.255.255.0
no shutdown
exit

ping 192.168.2.10

configure terminal
ip ips name IDS_RULE

interface fa0/0
ip ips IDS_RULE in
exit

write memory
show ip ips all
```

PC and Server configuration

```
PC0: Desktop → IP Configuration
IP Address: 192.168.1.10
Subnet Mask: 255.255.255.0
Default Gateway: 192.168.1.1

Server0: Desktop → IP Configuration
IP Address: 192.168.2.10
Subnet Mask: 255.255.255.0
Default Gateway: 192.168.2.1
```

Verification: From PC0, run **ping 192.168.2.10**. On the router, run **show ip ips all**. The source practical applies IDS_RULE inbound on FastEthernet0/0.

PRACTICAL NO. 9 — MALWARE ANALYSIS AND DETECTION

Topology / setup

```
Standalone Windows computer
          |
          v
  EICAR harmless test file
          |
          v
 Windows Defender / Antivirus
          |
          v
 Detection alert → Quarantine
```

The source practical does not specify a Packet Tracer network topology, router, switch, or cable connections. It is a workstation-based antivirus exercise. The flow above represents the analysis setup, not a physical network topology.

Tools and setup

Item	Use
Windows computer	Create and scan the test file
Windows Defender / Avast / Quick Heal / Norton	Antivirus scanning and detection
Task Manager	Observe CPU, memory and unknown processes
Command Prompt	Available system inspection tool
VirusTotal	Optional online malware scanner

Configuration / procedure

```
1. Open Notepad.
2. Enter this exact EICAR test string:
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
3. Save the file as: eicar.com

4. Open antivirus software.
5. Navigate to: Scan → Custom Scan
6. Select eicar.com and start scanning.
7. Observe the detection alert and quarantine action.

8. Open Task Manager and observe:
   CPU Usage
   Memory Usage
   Unknown Processes

9. Observe network activity and file activity.
10. Record malware type, detection name, risk level and behavior observed.
11. Apply countermeasures:
    Quarantine or delete the test file as appropriate.
    Update antivirus and run a full system scan.
    Enable firewall, install security updates and verify backups.
```

Expected observation in the supplied practical: File name eicar.com; threat detected EICAR-Test-File; action quarantined; risk level Test Threat (Harmless). The EICAR test file is a harmless antivirus test sample, not live malware.

PRACTICAL NO. 10 — FIREWALL CONFIGURATION AND RULE-BASED FILTERING

Topology

```
PC0  ----------------  Router 2811  ----------------  PC1
                         Fa0/0       Fa0/1

PC0:     192.168.1.10                PC1:     192.168.2.10
Router:  192.168.1.1 (Fa0/0)         Router:  192.168.2.1 (Fa0/1)
```

Connections

From device / port	To device / port	Cable
PC0 FastEthernet0	Router Fa0/0	Copper Straight-Through
Router Fa0/1	PC1 FastEthernet0	Copper Straight-Through

IP configuration

Device / interface	IP address	Subnet mask	Default gateway
PC0 FastEthernet0	192.168.1.10	255.255.255.0	192.168.1.1
Router Fa0/0	192.168.1.1	255.255.255.0	—
Router Fa0/1	192.168.2.1	255.255.255.0	—
PC1 FastEthernet0	192.168.2.10	255.255.255.0	192.168.2.1

Router configuration

```
enable
configure terminal

interface fa0/0
ip address 192.168.1.1 255.255.255.0
no shutdown

interface fa0/1
ip address 192.168.2.1 255.255.255.0
no shutdown

access-list 101 deny ip host 192.168.1.10 host 192.168.2.10
access-list 101 permit ip any any

interface fa0/0
ip access-group 101 in

show access-lists
show ip interface fa0/0
```

PC configuration

```
PC0: Desktop → IP Configuration
IP Address: 192.168.1.10
Subnet Mask: 255.255.255.0
Default Gateway: 192.168.1.1

PC1: Desktop → IP Configuration
IP Address: 192.168.2.10
Subnet Mask: 255.255.255.0
Default Gateway: 192.168.2.1
```

Testing

```
Before applying ACL:
From PC0 Command Prompt:
ping 192.168.2.10
Expected: Reply from 192.168.2.10

After applying ACL:
From PC0 Command Prompt:
ping 192.168.2.10
Expected: Request timed out.
```

The ACL blocks traffic from PC0 (192.168.1.10) to PC1 (192.168.2.10) and permits all remaining IP traffic. The ACL is applied inbound on Router Fa0/0.